Senior SDK Engineer, Native Core — Beam

About Surt

Surt AI is building the future of intelligent fraud detection and compliance. We develop advanced, data-driven systems that help companies identify fraud risks early, understand them clearly, and act with confidence.

Fraud signals are everywhere — turning them into meaningful, actionable insights requires clarity and speed. We act as a digital compliance officer, transforming complex fraud patterns into a clear and usable product experience so teams can focus on growth while staying compliant.

We are a small, focused team. We move fast, take ownership seriously, and build for scale.

About Beam

Beam is Surt's ID scanning SDK — a cross-platform native library that captures, processes, and extracts identity information from physical documents in real time. It runs on iOS, Android, and Web, operates fully on-device, and is designed to work on any hardware from flagship phones to 2GB-RAM budget devices in emerging markets.

Beam replaces third-party scanning dependencies — products like Scandit — with a system we own end to end: the camera pipeline, the frame selection logic, the ML inference integration, and the result parsing layer. Owning this stack is a strategic decision. The data Beam accumulates, the entity graph it feeds, and the intelligence it compounds over time are not things we can afford to outsource.

The Role

We are looking for a Senior SDK Engineer to own the native core of Beam. This is not an integration role — it is a build-from-first-principles role. You will design and implement the shared native core in C++ and Rust that runs across all three platforms, write the thin platform adapters in Swift and Kotlin, and own the performance characteristics of the full frame pipeline from camera interrupt to inference result. The language split is deliberate: C++ where the ML runtime boundary demands it, Rust where memory safety in business logic is the right tradeoff. You will decide where that line sits.

You will work directly with the Tech Lead and the ML team. You will make the architecture decisions that determine how fast Beam runs on a Helio G85 budget device, whether zero-copy frame delivery is maintained across HAL implementations, and whether the pipeline degrades gracefully when the ANE is unavailable. These decisions have direct consequences for the scan success rate in the field.

What You Will Own

- **Native shared core (C++ and Rust)** — the frame selection pipeline, quality gates (blur, exposure, motion), result parsing, session state management, and business logic that runs identically on iOS, Android, and WASM; C++ at the ML runtime boundary where zero-copy tensor input requires direct access to the TFLite and CoreML C++ APIs, Rust for the business logic layer where memory safety without GC is the right tradeoff

- **Platform adapters** — thin Swift (iOS) and Kotlin/JNI (Android) bridges that deliver zero-copy CVPixelBuffer and gralloc frame pointers into the native core; the WASM adapter that handles the mandatory copy boundary on web

- **Camera session configuration** — ISP-level format negotiation (YUV 4:2:0 NV12 natively), frame rate locking, buffer pool management, and HAL variance handling across the Android device fragmentation landscape

- **ML inference integration** — TFLite GPU delegate and NNAPI configuration on Android, CoreML and ANE dispatch on iOS, ONNX Runtime WASM with WebGPU on web; maintaining the zero-copy tensor input path from ISP DRAM to inference runtime

- **Performance across device tiers** — profiling and optimising the pipeline on 2GB-RAM budget devices (MediaTek Helio class) through to Apple A-series; ensuring the frame selection gates are correctly ordered and the ANE/GPU is not involved in frames that quality filters would reject

- **Cross-platform build system** — CMake for the C++ core, Emscripten for WASM, platform-specific packaging for iOS XCFramework and Android AAR; CI validation across all three targets

What We Are Looking For

- **Strong systems programming fundamentals in Rust or C++** — you understand memory layout, ownership semantics, cache behaviour, and the cost of every abstraction layer you introduce; you have shipped production code in at least one of these languages and can reason about the other without needing to learn it from scratch

- **Honest understanding of where each language fits** — the ML runtime boundary (TFLite, CoreML, ONNX Runtime) exposes a C++ API; zero-copy tensor input is only available through that API; you understand why this constrains the language choice at that specific layer and can make the call without ideology in either direction

- **Mobile native experience** — you have worked with AVFoundation or Camera2 at a level below the happy path; you know what CVPixelBuffer lock semantics mean, you have debugged gralloc buffer pool starvation, you have hit the JNI boundary and measured its cost

- **Familiarity with document scanning or computer vision SDKs** — experience with Scandit, BlinkID, Anyline, Google ML Kit, or similar is directly relevant; understanding why those products make the architectural choices they make tells us you have reasoned about this problem space at the right level

- **ML inference pipeline experience** — you have integrated TFLite, CoreML, or ONNX Runtime in a production mobile context; you understand the difference between CPU, GPU delegate, and NPU execution paths and when each is appropriate
- **Comfort with hardware-level reasoning** — you can think about what is happening on the memory bus, why cache pollution from a tensor copy matters on a 4MB L3 device, and what the ISP is doing before your code runs
- **Ownership orientation** — you will be given the problem and the constraints, not a specification; you are expected to produce the specification, validate it against reality, and own the outcome

What We Offer

- A technically deep role on a problem that has not been solved well at the intersection of performance, cross-platform correctness, and device-tier coverage
- Direct access to leadership and real influence over the architecture of a product that will process millions of identity documents
- A team that takes engineering seriously and values people who understand tradeoffs at the hardware level, not just the framework level
- Flexible working, competitive compensation, and the opportunity to own a critical piece of infrastructure from the ground up